

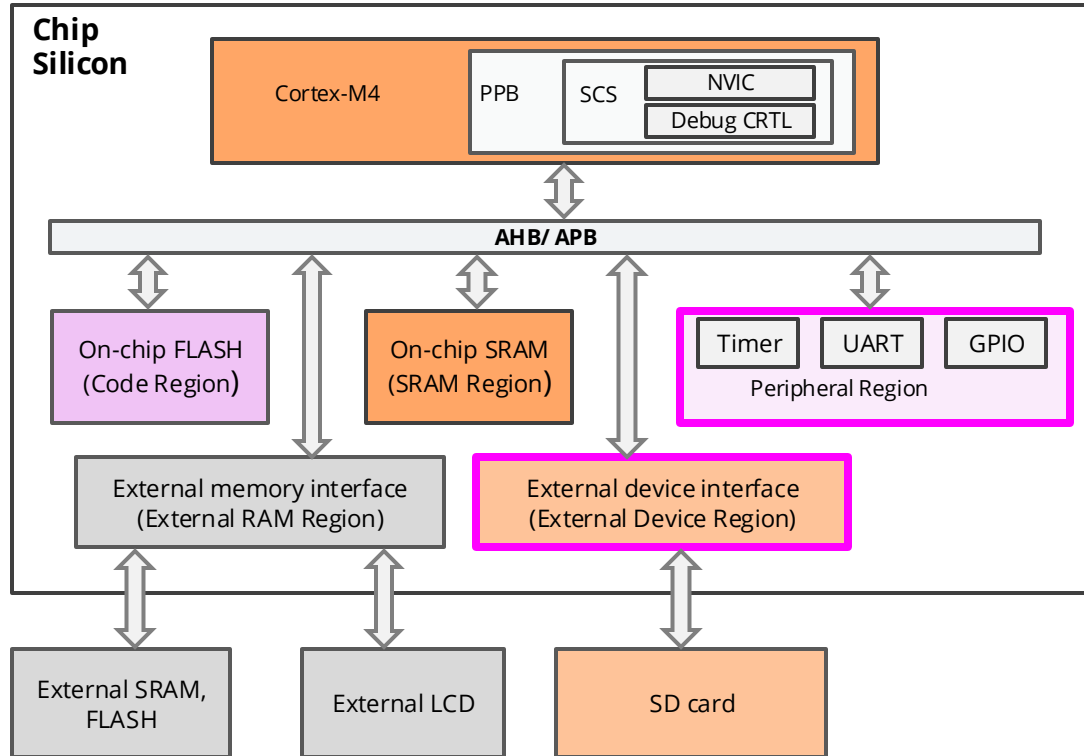


Embedded Software for the Internet Of Things

Processors and I/O

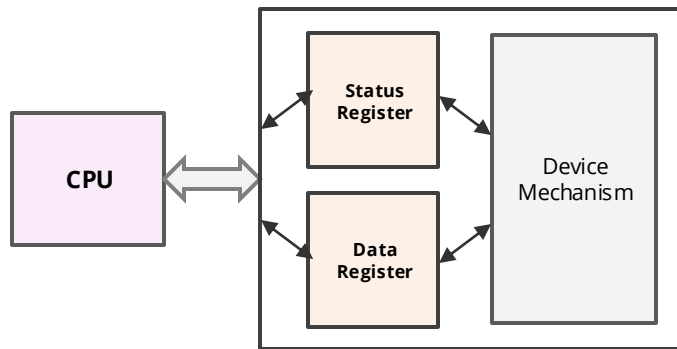
Assoc. Prof. Kasim Sinan Yildirim
University of Trento, Italy

Input and Output Devices



Programming Input and Output

- The CPU talks to the device by reading and writing **device registers**.
- Devices typically have several registers:
 - **Data registers**
 - *hold data values*
 - *can be readable or writable*
 - **Status registers**
 - *provide information about the device's operation*
 - e.g., the current operation has been completed
 - *can be read-only*



Input and Output Primitives

- Two ways to access those registers.
- **I/O instructions**
 - special instructions for input and output
 - ***in*** and ***out*** in the case of the Intel x86
 - provide a separate address space for I/O devices (separate address lines...)
- **Memory-mapped I/O**
 - Provides addresses for the registers in each I/O device
 - *programs use the CPU's normal memory read and write instructions to communicate with the devices.*

Example: Memory-mapped I/O on ARM

- Define location for device and provide Read/Write code:

```
DEV1 EQU 0x1000
```

```
LDR r1,#DEV1 ; set up device address
```

```
LDR r0,[r1] ; read DEV1
```

```
LDR r0,#8 ; set up value to write
```

```
STR r0,[r1] ; write value to device
```

Write I/O Devices in C

- We can use **pointers** to manipulate the addresses of I/O devices.
 - Introduce functions that read and write arbitrary memory locations:

```
int read(char *location) {  
    return *location;  
}  
  
void write(char *location, char newval) {  
    (*location) = newval;  
}
```

Write I/O Devices in C

- To read a device register we can write:

```
#define DEV1 0x1000  
...  
dev_status = read(DEV1); /* read device register */
```

- To write to the status register, we can use the following code:

```
write(DEV1,8); /* write 8 to device register */
```

Busy-Wait I/O

- I/O devices are typically slower than the CPU
 - They may require many cycles to complete an operation.
- **Busy-wait**
 - Asking an I/O device whether it is finished by reading its **status register**
 - Often called **polling**.

Busy-Wait I/O (Example)

- Write a sequence of characters to an output device.
 - The device has two registers:
 - *one register for the character to be written (data register),*
 - *a status register.*
 - The status register's value is:
 - *1: when the device is busy writing*
 - *0: when the write has been completed.*

Busy-Wait I/O (Example)

- Send the characters and wait for each transaction to complete:

```
#define OUT_CHAR 0x1000    /* output device character register */
#define OUT_STATUS 0x1001 /* output device status register */
...
char *mystring = "Hello, world." /* string to write */
char *current_char;              /* pointer to the current position in string */
current_char = mystring;        /* point to the head of string */

while (*current_char != '\0') { /* until null character */
    write(OUT_CHAR, *current_char); /* send character to device */
    while (read(OUT_STATUS) != 0); /* keep checking status */
    current_char++;                /* update character pointer */
}
```

Busy-Wait I/O (Example)

- Repeatedly read data from the input device and send to the output device.
- When a new character has been read:
 - the input device sets its status register to 1
 - *we must set it back to 0 so that the device is ready to read another character*
- To start writing:
 - first set the output status register to 1,
 - then wait for it to return to 0

Busy-Wait I/O (Example)

- Repeatedly perform the read/write operation:

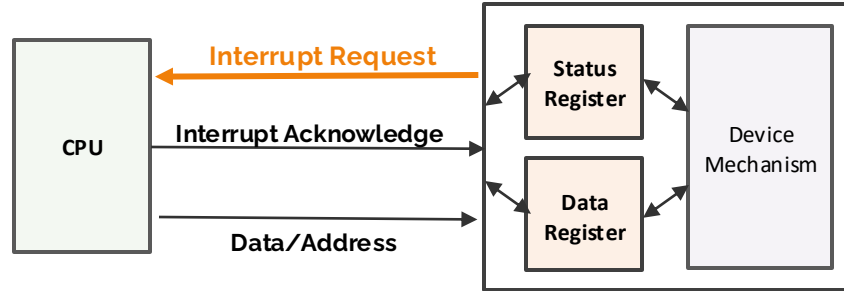
```
#define IN_DATA 0x1000
#define IN_STATUS 0x1001
#define OUT_DATA 0x1100
#define OUT_STATUS 0x1101
...
while (TRUE) { /* perform operation forever */
    /* read a character into achar */
    while (read(IN_STATUS) == 0); /* wait until ready */
    achar = (char)read(IN_DATA); /* read the character */
    write(OUT_DATA,achar); /* write achar */
    write(OUT_STATUS,1); /* turn on device */
    while (read(OUT_STATUS) != 0); /* wait until done */
}
```

Interrupts

- Busy-wait I/O is **inefficient**.
 - The CPU could do useful work in parallel with the I/O
- The interrupt mechanism allows devices to **signal** the CPU to force the execution of a particular piece of code.

Interrupts

- Several signals control the interrupt process.
- When the I/O device wants service from the CPU:
 - It asserts the **interrupt request signal**
- When the CPU is ready to handle the I/O device's request
 - It asserts the **interrupt acknowledge signal**



Interrupts

- The I/O device's logic decides when to interrupt
 - e.g., when data is ready (user pressed a button)
- When the CPU decides to acknowledge the interrupt:
 - It changes the program counter (like a subroutine call) to point to the device's **interrupt handler**

Interrupts

- Therefore, interrupts change the **control flow** in the CPU.
- When an interrupt occurs:
 - First, the value of the PC at the interruption is saved:
 - *the CPU can return to the **foreground program** later*
 - Then, the PC starts pointing to an **interrupt handler** routine:
 - *this routine takes care of the device by reading data that have just become ready, writing the next data...*

Interrupts (Example)

- Copying Characters From Input to Output With Basic Interrupts
 - **achar**: passes the character to the foreground program.
 - **gotchar**: signals when a new character has been received.

INTERRUPT HANDLERS

```
/* get a character and put in global (called when IN_STATUS is 1) */
void input_handler() {
    achar = read(IN_DATA); /* get character */
    gotchar = TRUE;        /* signal to main program */
    write(IN_STATUS,0);    /* reset status to initiate next transfer */
}

/* react to character being sent (called when OUT_STATUS is 0) */
void output_handler() {
    /* don't have to do anything */
}
```

Interrupts (Example)

- Copying Characters From Input to Output With Basic Interrupts
 - The main program somewhat simpler compared to Busy Wait I/O
 - But still does not let the foreground program do useful work.
 - It *polls* input and writes data...

```
main() {  
    while (TRUE) { /* read then write forever */  
        if (gotchar){ /* write a character */  
            write(OUT_DATA,achar); /* put character in device */  
            write(OUT_STATUS,1);    /* set status to initiate write */  
            gotchar = FALSE;        /* reset flag */  
        }  
    }  
}
```

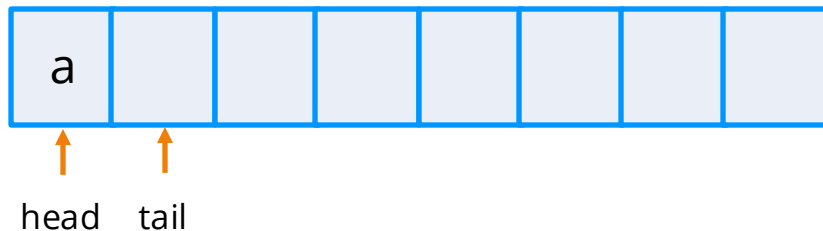
Interrupts (Example)

- Copying Characters From Input to Output With Interrupts and Buffers
 - We will use a **circular** (wraparound) buffer to hold the characters.



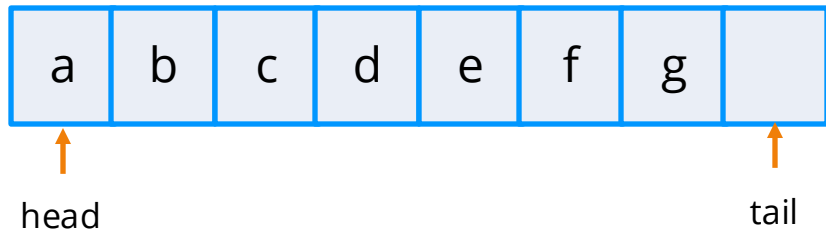
Interrupts (Example)

- Copying Characters From Input to Output With Interrupts and Buffers
 - We will use a **circular** (wraparound) buffer to hold the characters.



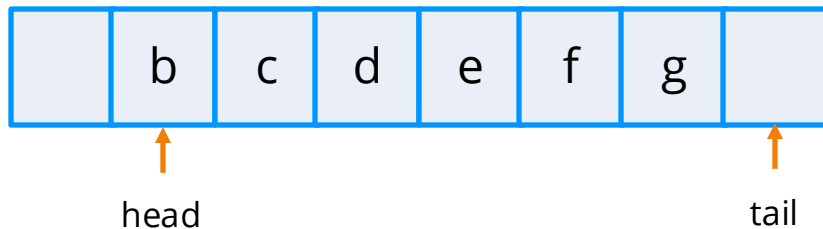
Interrupts (Example)

- Copying Characters From Input to Output With Interrupts and Buffers
 - We will use a **circular** (wraparound) buffer to hold the characters.



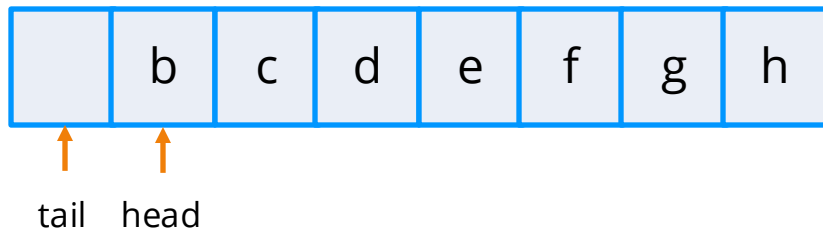
Interrupts (Example)

- Copying Characters From Input to Output With Interrupts and Buffers
 - We will use a **circular** (wraparound) buffer to hold the characters.



Interrupts (Example)

- Copying Characters From Input to Output With Interrupts and Buffers
 - We will use a **circular** (wraparound) buffer to hold the characters.



Interrupts (Example) - With Circular Buffer

```
#define BUF_SIZE 8
char io_buf[BUF_SIZE];           /* character buffer */
int buf_head = 0, buf_tail = 0; /* current position in buffer */
int error = 0;                   /* set to 1 if buffer ever overflows */

int empty_buffer() { /* returns TRUE if buffer is empty */
    return buf_head == buf_tail;
}

int full_buffer() { /* returns TRUE if buffer is full */
    return (buf_tail+1) % BUF_SIZE == buf_head ;
}

int nchars() { /* returns the number of characters in the buffer */
    if (buf_head >= buf_tail)
        return buf_head - buf_tail;
    else
        return BUF_SIZE - buf_tail - buf_head;
}
```

Interrupts (Example) - With Circular Buffer

```
void add_char(char achar) { /* add a character to the buffer head */
    io_buf[buf_tail++] = achar;
    /* check pointer */
    if (buf_tail == BUF_SIZE)
        buf_tail = 0;
}

char remove_char() { /* take a character from the buffer head */
    char achar;
    achar = io_buf[buf_head++];
    /* check pointer */
    if (buf_head == BUF_SIZE)
        buf_head = 0;

    return achar;
}
```

Interrupts (Example) - With Circular Buffer

```
#define IN_DATA 0x1000
#define IN_STATUS 0x1001

/* get a character (called when IN_STATUS is 1) */
void input_handler() {
    char achar;
    if (full_buffer()) /* error */
        error = 1;
    else { /* read the character and update pointer */
        achar = read(IN_DATA); /* read character */
        add_char(achar); /* add to queue */
    }
    write(IN_STATUS,0); /* set status register back to 0 */

    /* if buffer was empty, start a new output transaction */
    if (nchars() == 1) { /* buffer had been empty until this interrupt */
        write(OUT_DATA,remove_char()); /* send character */
        write(OUT_STATUS,1); /* turn device on */
    }
}
```

Interrupts (Example) - With Circular Buffer

```
#define OUT_DATA 0x1100
#define OUT_STATUS 0x1101

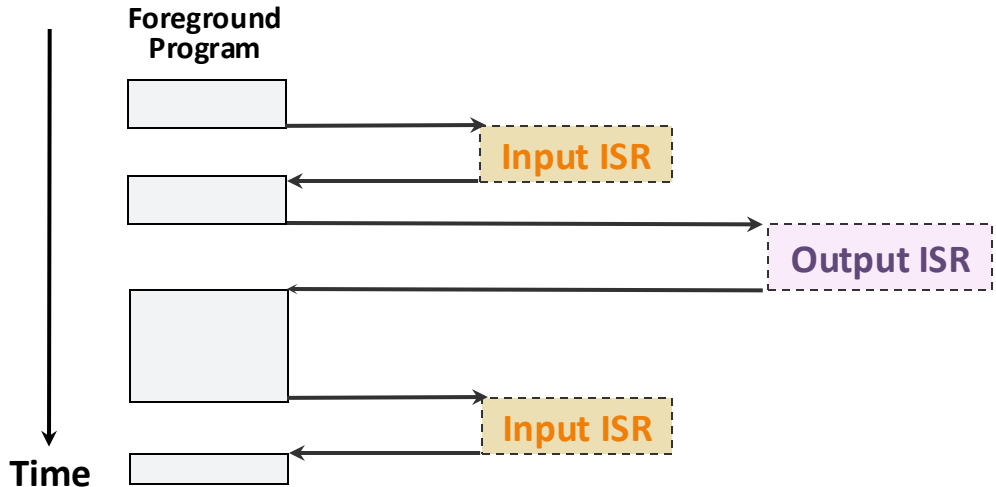
/* react to character being sent (called when OUT_STATUS is 0) */
void output_handler() {
    if (!empty_buffer()) {                                /* start a new character
    */
        write(OUT_DATA,remove_char());/* send character */
        write(OUT_STATUS,1);          /* turn device on */
    }
}
```

Interrupts (Example) - With Circular Buffer

- The **foreground program** is occasionally interrupted by input and output operations:
 - *all input and output operations are handled by the ISRs **in the background!***
 - The foreground program continues to run in its regular state after interrupt handlers

Foreground Program

```
void main(){  
...  
for (i = 0; i < M; i++) {  
    y[i] = b[i];  
    for (j = 0; j < N; j++)  
        y[i] = y[i] + A[i,j]*x[j];  
}  
...  
}
```

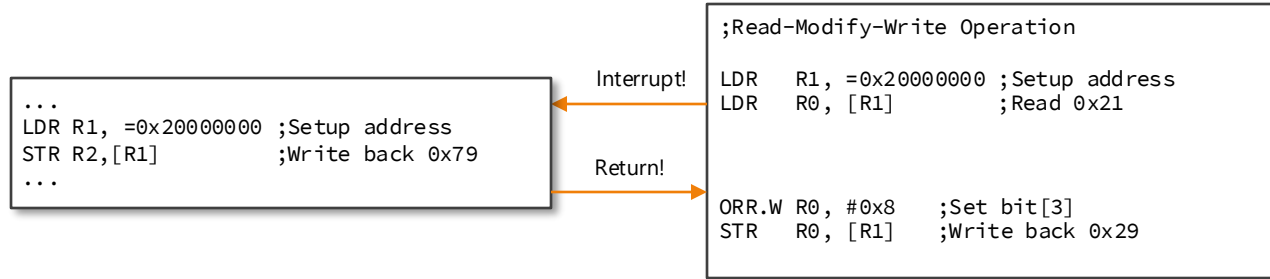


Bugs in Interrupt Code

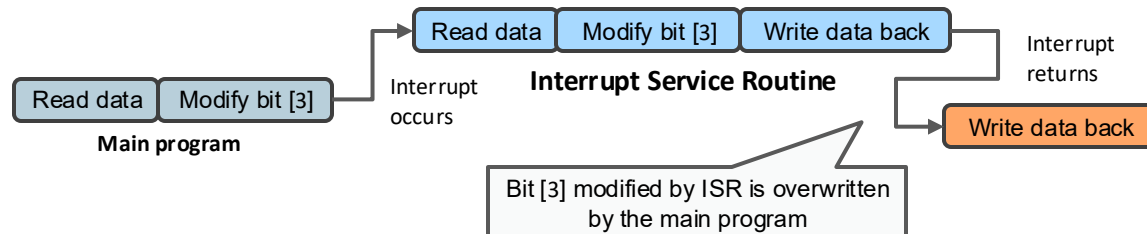
- The errors can be very hard to find when the interrupt handlers are buggy due to the **concurrency**
- Interrupt handler:
 - must save any CPU register that it will modify
 - must restore them before it exits
- Forgetting to save/restore a register in the handler might cause that register to **mysteriously change a variable** in the foreground program.

Bugs in Interrupt Code

- For example, in order to set **bit[3]** in **word data** in address **0x20000000**:

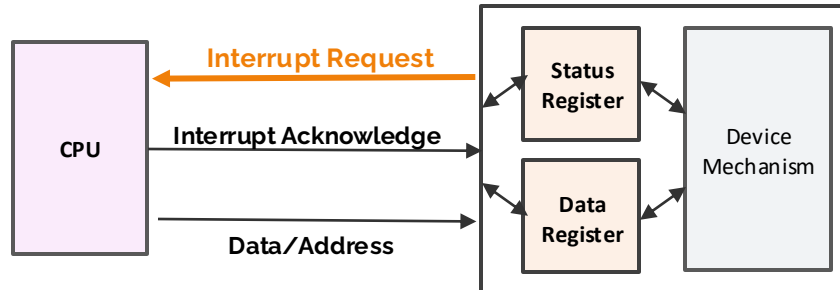


- Read-modify-write operation**
 - Reads the data (0x21) from the address 0x20000000
 - The interrupt changes the data of 0x20000000 address to 0x79 and then returns
 - Writes back the modified old data back
 - 0x79 has been lost!



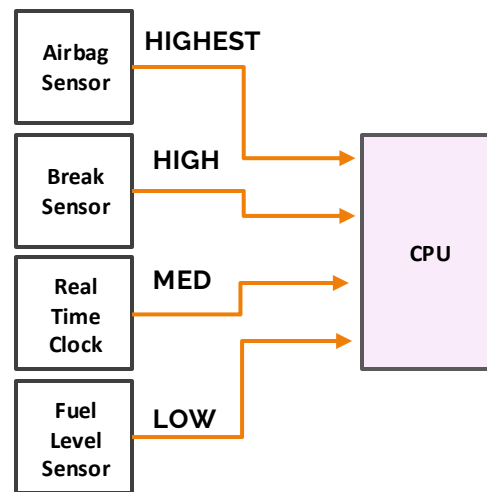
Interrupts - Implementaton

- The CPU checks the **interrupt request (IRQ)** line at every instruction.
- If an interrupt request has been asserted, the CPU:
 - puts the return address on a stack (as CPU does for subroutines)
 - does not fetch the instruction pointed to by the PC
 - *sets the PC to the beginning of the interrupt handler*
- The address of the interrupt handlers are stored in a table.



Priorities and Vectors

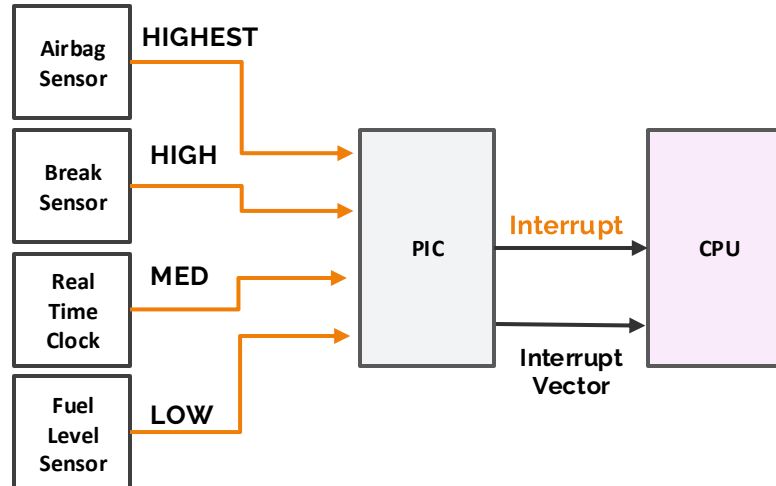
- Most systems have more than one I/O device:
 - multiple devices can interrupt
 - several interrupt handlers, device addresses
- **Interrupt priorities**
 - recognize some interrupts as more important than others
- **Interrupt vectors**
 - allow the interrupting device to specify its interrupt handler



Priorities and Vectors

- **Programmable interrupt controller (PIC)**

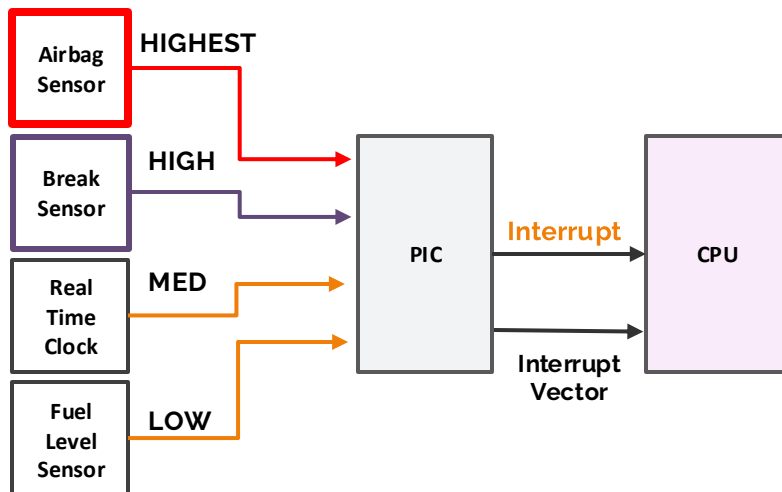
- **Prioritize** multiple interrupt sources so that at any time the highest priority interrupt is presented to the core CPU for processing.



Priorities and Vectors

- **Nested interrupts**

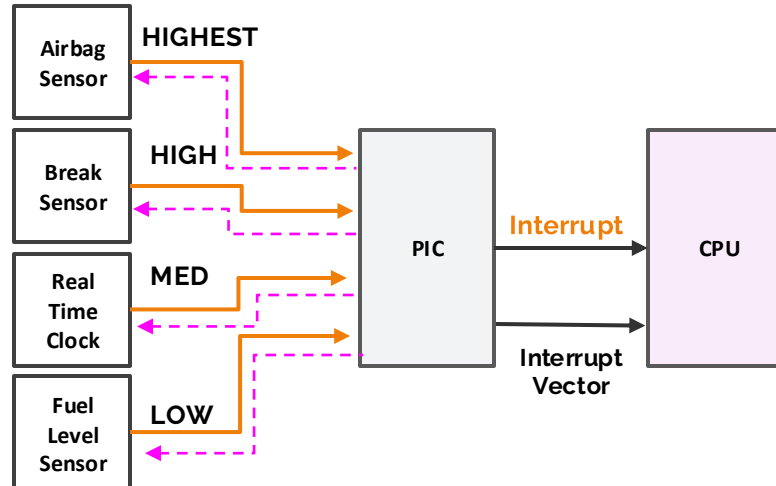
- a higher priority interrupt source to **preempt** the processing of a lower priority interrupt.



Priorities and Vectors

- **Interrupt acknowledge:**

- A device knows that its interrupt request was accepted by seeing its own priority number on the interrupt acknowledge lines



Priorities and Vectors

- **Interrupt Masking:**

- ensures that a lower-priority interrupt does not occur when a higher-priority interrupt is being handled.

- When an interrupt is acknowledged:

- CPU stores the **priority level** of that interrupt in an internal register

- When a subsequent interrupt is received:

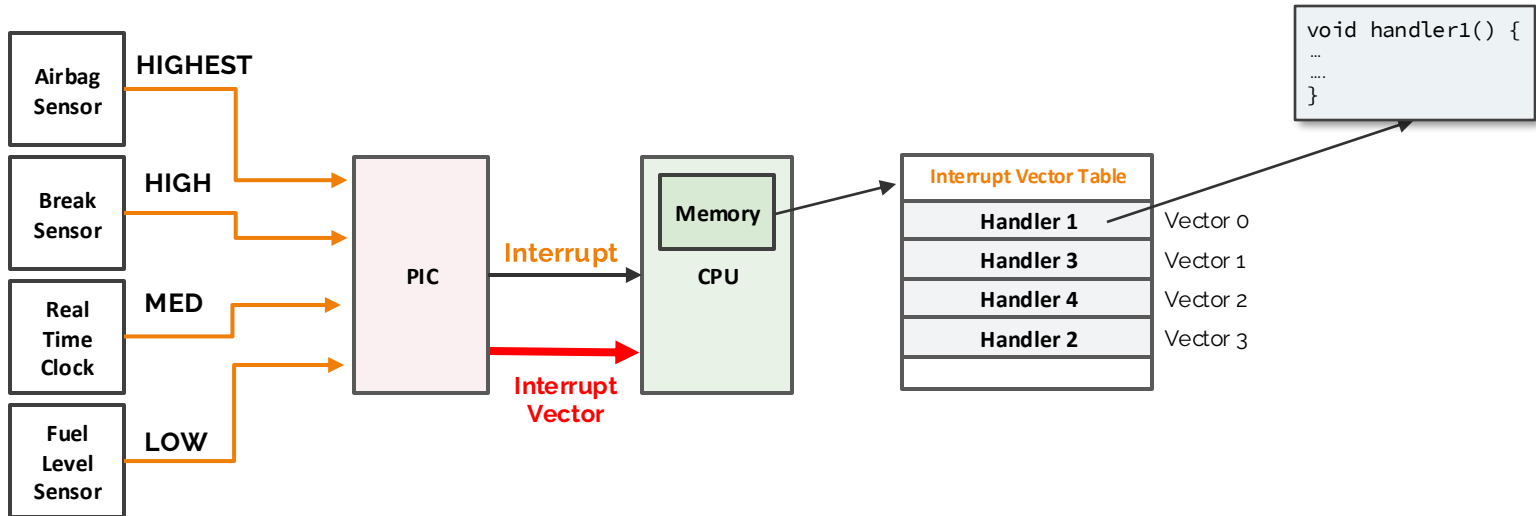
- its priority is checked against the priority register and it is acknowledged only if it has a higher priority

Priorities and Vectors

- The highest-priority interrupt is called the **nonmaskable interrupt (NMI)**.
- The NMI cannot be turned off
 - e.g., usually reserved for interrupts caused by power failures to save critical state in nonvolatile memory, turn off I/O devices...

Priorities and Vectors

- After the device is acknowledged, it sends its **interrupt vector number**
 - An index for the **interrupt vector table** stored in memory.
 - **Interrupt vector**: the memory address of the interrupt handler.



Overhead of Interrupts

- An interrupt causes a change in the program counter:
 - incurs a **branch penalty**
- Interrupt might automatically stores some CPU registers:
 - requires **extra cycles**
- Acknowledge the interrupt and obtain the vector from the device
 - requires **extra cycles**
- The **interrupt handler overhead**:
 - Saves and restores CPU registers that were not automatically saved
 - The **interrupt return instruction** restores the automatically saved state, incurs a **branch penalty**

Overhead of Interrupts

- **Interrupt Response Time:**

- The time required for the hardware to respond to the interrupt, obtain the vector, save state and so on **cannot be changed by the programmer**.

